ME 588: Project Report
Team 10
Storm that Castle

Christian Scott; David Lu; Gurman Singh; Heng Zheng; Jau-Shiuan Shiao; Leonardo Franquilino

May 4^{th}, 2026

Abstract

This document describes the strategy, design, and commissioning of the robot developed by Team 10 to compete in the "Storm the Castle" competition for ME588. The primary objective of the competition is to throw balls into buckets controlled by the opposing team. The maximum robot size, minimum design requirements, and all competition rules were established by the course instructor.

Accordingly, this document focuses on describing the team's overall strategy, chassis and motion system design, component selection, trebuchet design, control strategy, finite state machine implementation, robot assembly process, results obtained during check-offs, and a discussion of both the team's successes and the challenges encountered. The final section presents the overall conclusions.

Overall, the team demonstrated strong collaboration, and all members contributed significant time and effort throughout the project. The robot successfully met the minimum competition requirements and was able to complete a run in which it launched a ball into the opposing team's bucket. However, the team encountered several commissioning challenges, particularly related to component calibration and system integration.

Introduction

The overall strategy agreed between the team members was to design a robot to follow the line present in the field, parallel to the buckets back and forth, without having to spin the robot. When identifying a crossline, an IR sensor reads the bucket status, and in case the allegiance of the bucket is from the enemy, a ball is thrown using the trebuchet mechanism. This overall idea is shown in Figure 1

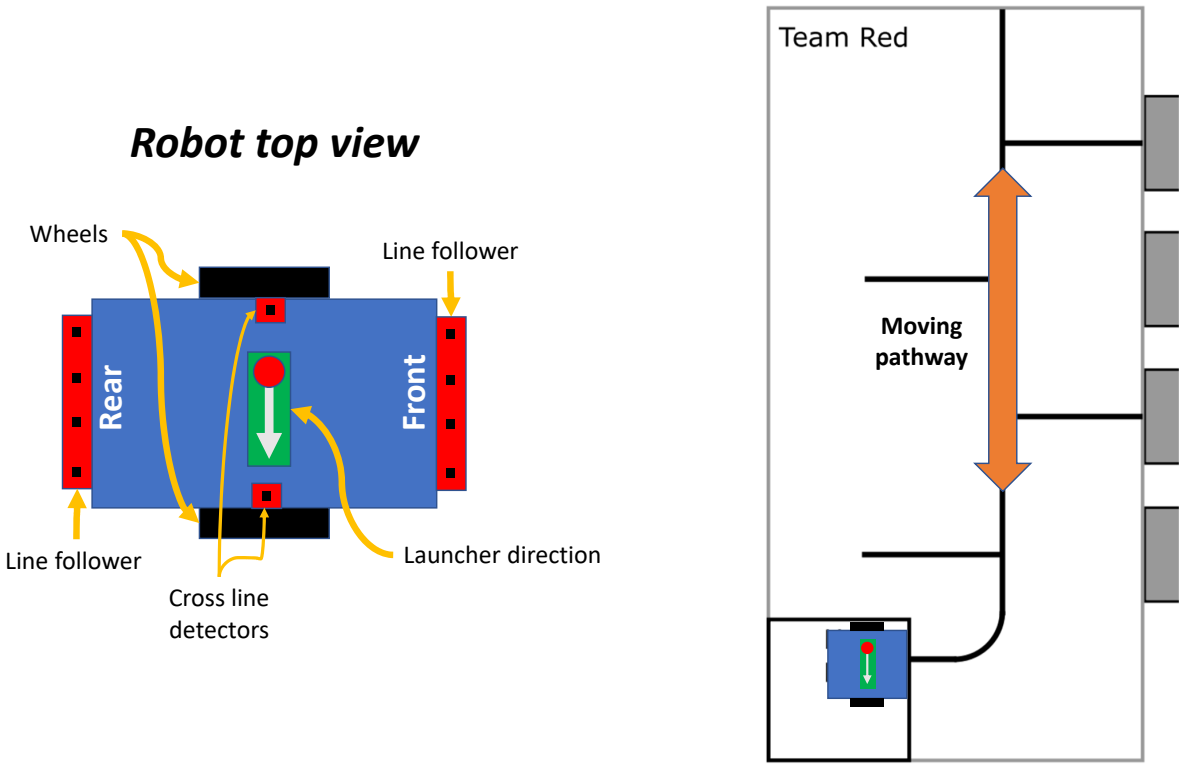

Figure 1: Robot fighting strategy

The sensors integrated into the robot are:

- 10 total reflectance sensors: 2 arrays of 4 for line following and 2 for line-crossing detection
- A break-beam sensor to detect whether a ball is loaded in the robot
- An IR sensor to measure the LED blinking frequency

The actuators integrated into the robot are:

- Two DC motors for wheel actuation
- Two servo motors: one for loading the trebuchet and the other for actuating the shooting mechanism

In addition to the sensors and actuators, several other components are integrated into the robot. These include a push button for game start, an encoder switch (used as a push button) for team color selection, an emergency stop switch capable of cutting power to the entire system, a green LED that indicates when the robot is in game mode, the on board RGB LED of the ESP-32 to display the selected team color (blue or red), and a white LED to indicate when the shooting mechanism is active.

The robot is controlled by an Espressif Systems ESP32-S3 microcontroller, which serves as the system's central processing unit. This microcontroller was selected due to its high processing capability, operating

at up to 240 MHz, integrated Wi-Fi and Bluetooth connectivity, and a total of 45 programmable GPIO pins. Additionally, it offers a lower cost compared to many Arduino boards while providing greater computational performance and flexibility.

IR sensor & circuitry

To determine the current allegiance of the HILLs, the robot must accurately detect and process the infrared (IR) beacons emitted from the targets. The team selected the TEKT5400S, a silicon NPN phototransistor whose spectral bandwidth closely matches the emission wavelength of the HILL IR LEDs. Because the raw voltage output from the phototransistor is inherently low and decays proportionally as detection distance increases, the signal requires significant conditioning before it can be read by the microcontroller. Specifically, a robust waveform with strong voltage levels and sharply defined edges is necessary to trigger the hardware interrupts on the ESP32-S3, enabling precise calculation of the 750 Hz and 1500 Hz target frequencies.

To achieve this, the team designed a custom two-stage signal conditioning circuit to amplify the raw signal, filter environmental noise, and digitize the waveform (Figure 2). Both operational amplifiers operate on a 5V single supply. The first stage utilizes an inverting amplifier configuration with a 2.5V virtual ground reference to provide the initial gain. The second stage functions as a comparator, transforming the amplified analog signal into a discrete digital pulse train.

Furthermore, high-pass and low-pass filters are integrated into the architecture to ensure signal integrity. This circuit design yields the following operational features:

- Targeted Bandpass Filtering: Frequencies below 160 Hz and above 1600 Hz are significantly attenuated. This effectively isolates the operational beacon frequencies while rejecting ambient IR noise, such as 60 Hz flicker from overhead lighting.
- Logic-Level Compatibility: The comparator outputs a clean square wave oscillating between 0V and 3.4V, which safely and favorably interfaces with the 3.3V logic pins on the ESP32-S3 microcontroller.
- Extended Detection Range: The two-stage amplification ensures a robust, reliable square wave output even when the robot is scanning from a distance of up to 6 feet away from the HILL.

Figure 3 exhibits the physical representation of the circuit assembled on a breadboard, while Figure 4 illustrates the resulting clean square waveform captured on an oscilloscope.

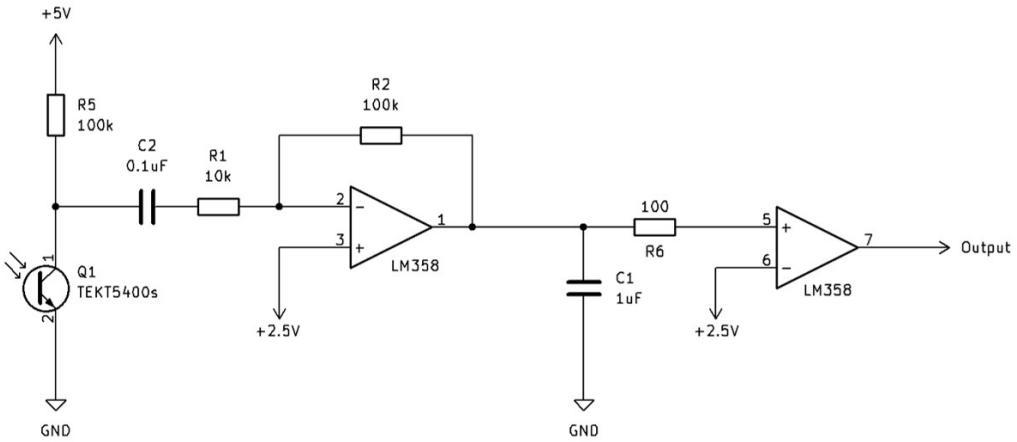

Figure 2: Schematic of the two-stage IR signal conditioning and amplifying circuit.

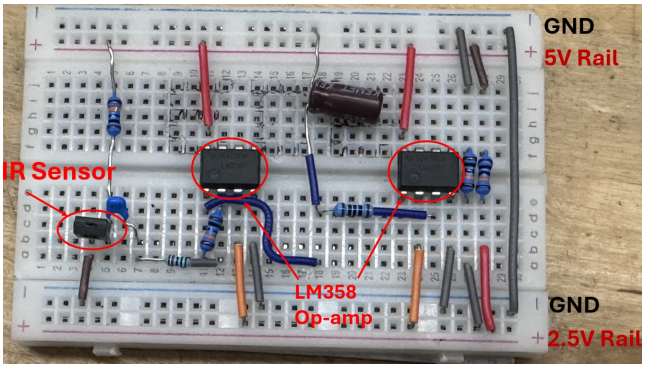

Figure 3: Physical assembly of the IR signal conditioning circuit on a breadboard.

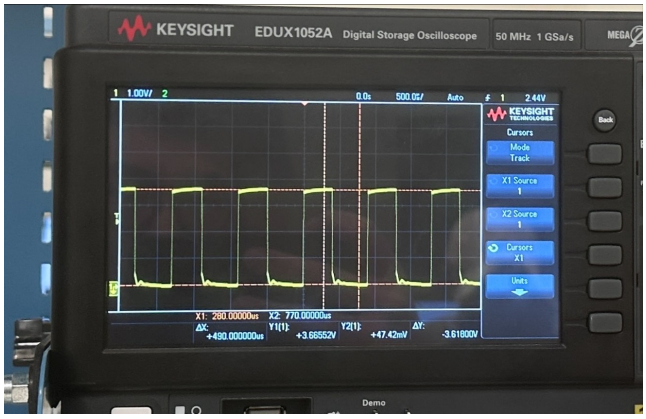

Figure 4: Oscilloscope captures clean square wave output from the conditioning circuit.

Launcher

The launcher was originally planned to use a flywheel system, but after testing it was found that the design used too much power and was not very consistent. Since the balls can have slight variations in size and surface, the flywheel did not always grip the same way, which caused inconsistent shots. Because of this, a decision was made to move away from the flywheel design and instead use a rail gun style system that would give more controlled and repeatable performance. This new design is shown in the figures below.

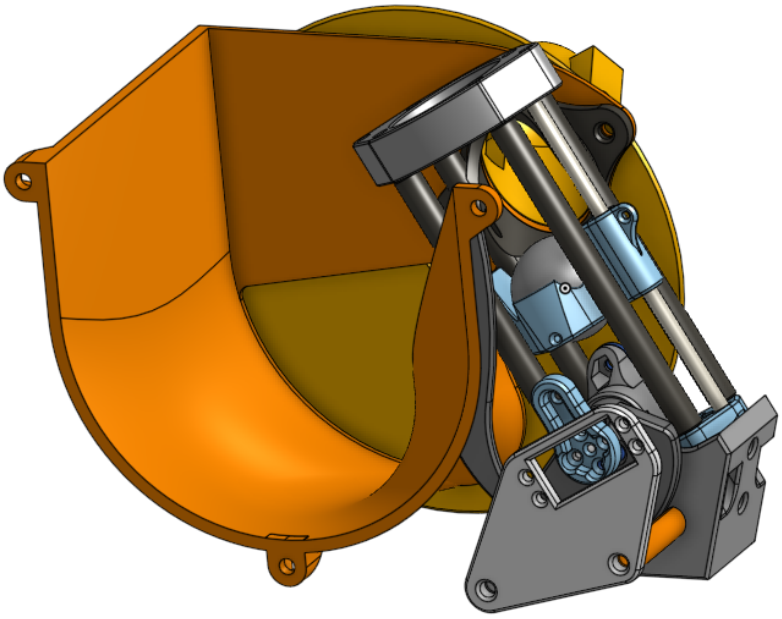

Figure 5: New Launcher Mechanism

The launcher is controlled by a servo, where the servo pulls the cradle back and then releases it to shoot. The design is similar to a rail gun style system that uses a ratchet mechanism. The ratchet has a toothed rack and a spring-loaded pawl that only allows motion in one direction and prevents it from slipping back. This lets the launcher hold its position when pulled back and then release consistently when

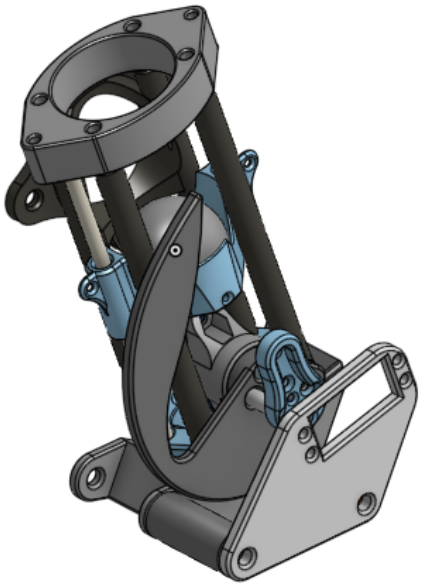

Figure 6: Launcher Design

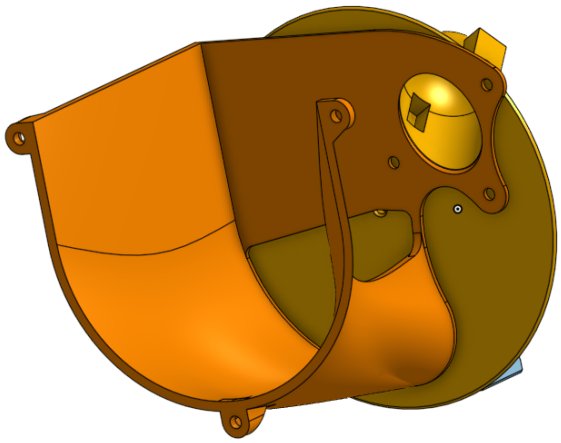

Figure 7: Hopper Design

needed. This setup allowed shooting from different ranges, and instead of using fixed positions, the servo was tuned in the program to be able to adjust how far the cradle is pulled back and when it releases for adjustability.

For feeding the ball, a side-loading system was used with rails that guide the ball into place. The hopper is slightly slanted, so the balls naturally roll down into the loader catch. The loader plate rotates and picks up a ball, then moves it into the launcher when it is ready to shoot. A break beam sensor detects if a ball is loaded, since the beam is broken when the ball is in position and not broken when it is empty. The motion of the loader and timing of the system were tuned in code instead of locking in exact angles, which made it easier to adjust to improve the reliability in loading and shooting.

Chassis & Motion

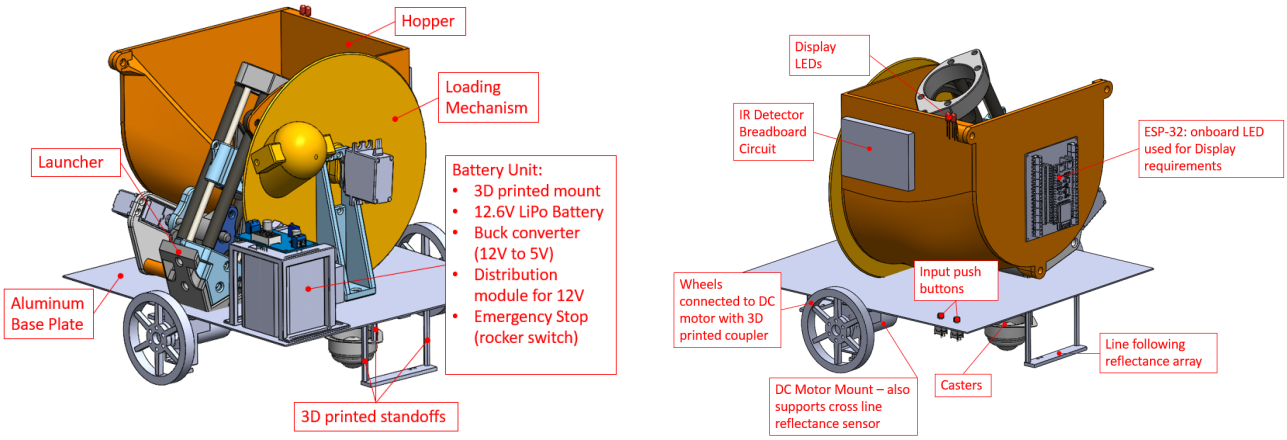

Figure 8: Final 3D Model

Figure 8 outlines the planned final assembly with the sub-components labeled. Initially, a pre-made base plate was going to be used, but to gain more space, a larger aluminum base plate was fabricated. As mentioned, two 12V motors were used to drive the robot with the casters providing additional stability. The motors were mounted using a 3D printed mount that also housed the cross line sensors which were placed in line with the launcher to align the launcher with the HILLs. With these cross line sensors and a coded counter, the robot knows which junctions it is between at all times. The motor driver (L298 H-bridge), not visible in Figure 8, was mounted underneath the aluminum base plate to save space on the top surface. For directionality, two line following arrays were used to strafe on the line. A 3D printed battery mount was used to consolidate the entire power system - the buck converter, the distribution module, and the emergency stop (rocker switch).

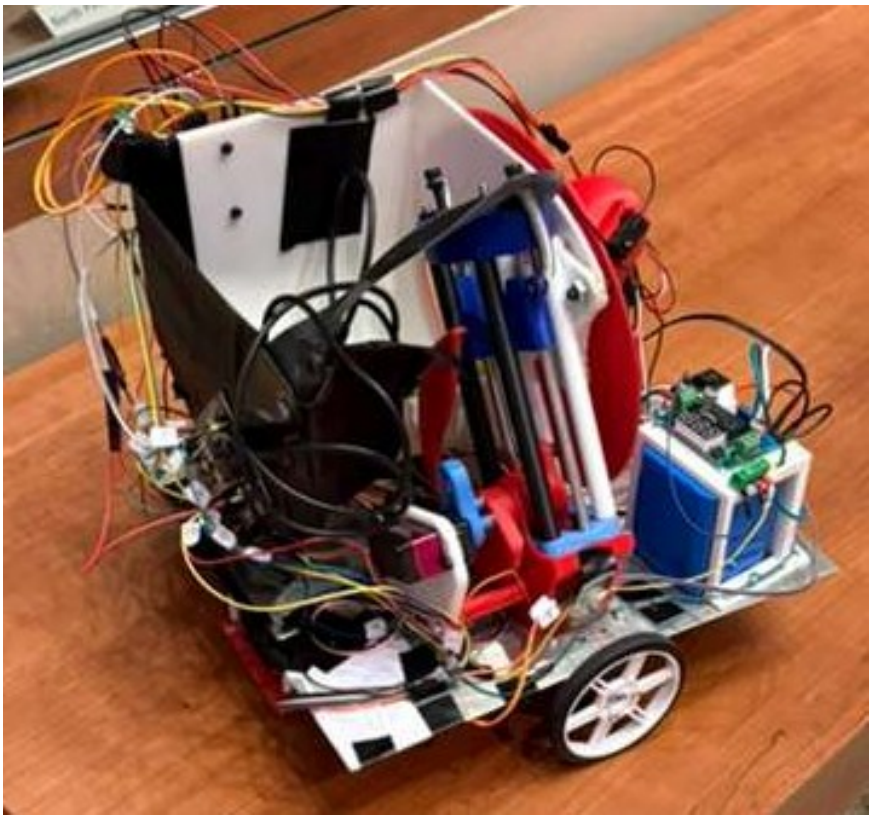

Figure 9: Final Assembly

Figure 9 shows the final assembly as seen on the night of the competition. Due to a printing issue with the hopper retention plate, tape was used to complete the hopper. This still allowed for mounting the ESP32 in this area. Other than this printing issue, the final assembly matches the final planned arrangement.

Control Strategy & Finite State Machine

The finite state machine (FSM) stores the current state, beacon state, team, location, and heading variables. In MoveToNextState, the robot will keep following the line using PI control. By keeping the location information, the robot could tell whether it passes the last junction in the end of the corridor, and if the heading is forward, it will change heading to backward. Similarly, when it identifies the location is at junction 1 and the heading is backward, it will switch heading to forward. This way, the robot will keep navigating between junction 1 and junction 4.

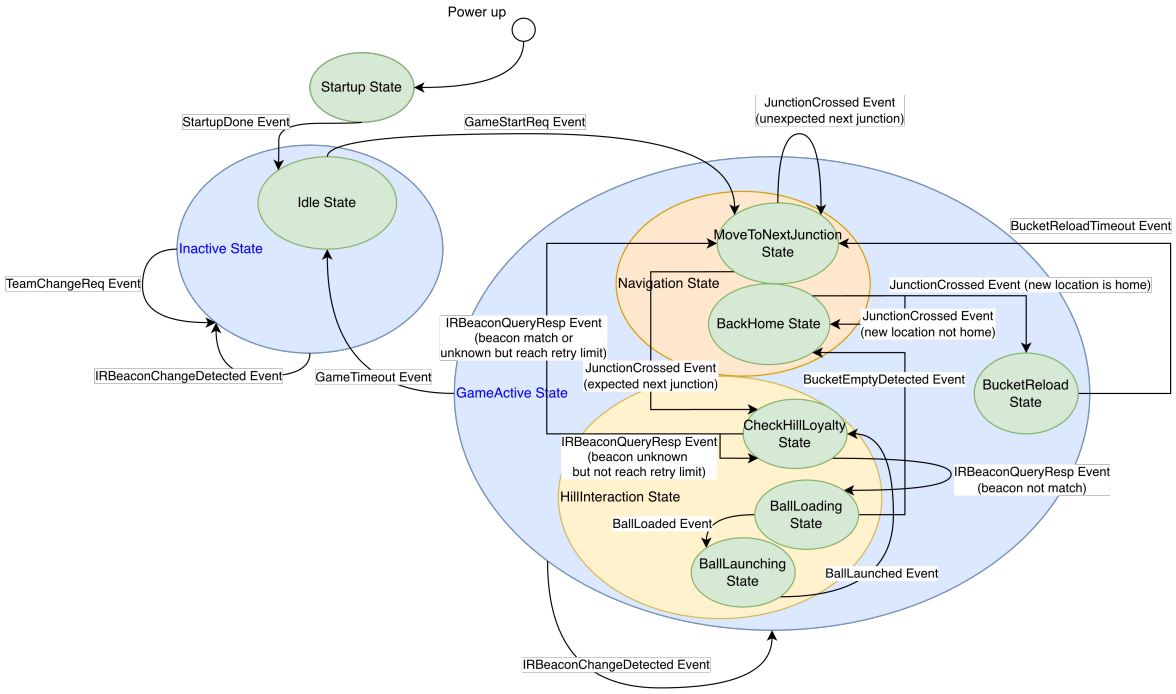

Figure 10: Hierarchical event-driven FSM

RTOS tasks

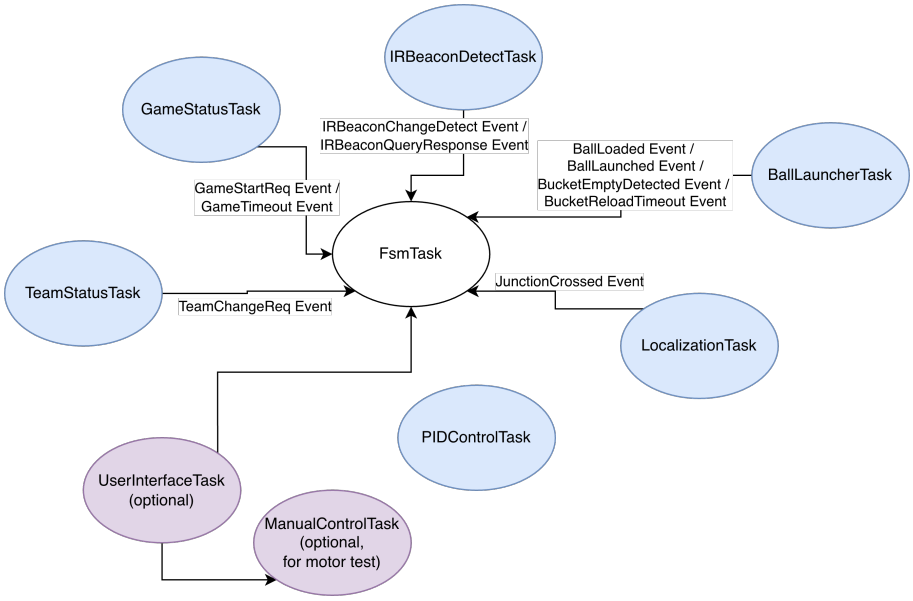

Figure 11: freeRTOS tasks

Results

For the results, all required checkoffs were successfully completed, including the design, circuit, and final demonstrations. During these checkoffs, the robot performed as expected. The launcher was able to shoot consistently, the IR detection system functioned properly, and the robot met the 12 x 12 x 12 inch

size constraint. The display requirements were also demonstrated, confirming that the mechanical design and hardware integration were solid.

During the competition, the robot was able to drive toward the hill and attempt to shoot, but issues occurred with the line following system. The sensors were not consistently detecting the tape and had difficulty at cross junctions, which affected positioning. As a result, even though the launcher itself functioned, the robot was not able to properly align with the hill and successfully score during the run.

Overall, the robot performed well and most subsystems worked as intended. The main limitation came from the coding and sensor tuning, particularly with line following and navigation. With additional debugging and refinement of the control logic, the robot would have been able to operate fully as intended during the competition.

Discussion

The team made strong progress at the beginning of the project, although several design changes were required throughout the development process. For example, the initial concept presented during the first check-off included an I2C display and a flywheel-based shooting mechanism. However, based on instructor feedback regarding display visibility from a distance of 6 feet, the display system was changed to LEDs only.

The shooting mechanism was also redesigned, transitioning from a flywheel-based system to a spring-loaded mechanism. This change was motivated by shooting consistency, as the spring-loaded design demonstrated more reliable and repeatable performance compared to the flywheel approach.

Regarding the finite state machine (FSM), development was initially started in MATLAB Simulink. However, Jau-Shiuan demonstrated greater experience and expertise in C/C++, and as a result, development in Simulink was discontinued in favor of direct implementation in code.

Conclusion

For the "Storm the Castle" project, a robot was designed to follow the base line and shoot in the buckets that were not owned by the assigned team. While the final robot was not able to follow the line as planned, further tuning and testing of the reflectance sensors and PI control would likely allow for a fully functioning robot as every other subsystem - launching, IR detection, and display - operated as intended. Further testing would focus on tuning the threshold of the reflectance sensors for the game surface and improving on the current PI controller.